

TickTrade — engineering report for the development team

Prepared 14 September 2026 · covers every change delivered since the 12 September 2026 code drop · backend `ASPR-backend-`, admin `APSR-admin-`, trading app `Tick-Trade-project-APSR`, and the owner's local tool "TickTrade Intelligence Core"

How to read this document

Each chapter describes one area: what was wrong (with the evidence that showed it), what changed, which files carry the change, and how it was verified. Every change also exists as a git patch (`patches/<repo>/NNNN-*.patch`) and as a commit in the delivered ZIPs, so the diff is always the final word; this document explains the intent behind it.

All verification was done on a complete local copy of the stack ("the workbench"): PostgreSQL 16, Redis 7, the API, the battle engine, the engine worker, the settlement worker, the NPC worker, the trading PWA and the admin app, built from the same sources that were shipped, with browser checks driven by headless Chromium. Where a number is quoted (pips, dollars, counts) it was measured there.

Chapters:

1. Trading app (PWA)
2. Admin app — login, dashboard, fonts, CI
3. Backend — CORS, Swagger, email tests, risk-terminal tick, NPC OFF flag
4. Organic NPC engine (the new NPC architecture)
5. Market Impact & Sensitivity
6. Admin risk and inspection tools (Risk Breaches, Risk Management Terminal, Price Movement Audit, Battle Inspector)
7. TickTrade Intelligence Core (the local tool)
8. Deploying, testing and known pre-existing failures

1. Trading app (PWA)

1.1 Trade confirmation sheet honours the Settings toggle (frontend patch 0004)

Wrong. `User.tradeConfirmation` was stored by the Settings screen but never read: real-money BUY and SELL executed immediately whatever the toggle said.

Changed. A confirmation sheet (`src/components/TradeConfirmSheet.jsx` + CSS) opens before a real-money order when the toggle is on; Cancel places no stake, Confirm sends the same socket order as before. Free Tap is unaffected. Wired in `src/components/TradingPanel.jsx`.

Verified. Browser check: with the toggle on, BUY opens the sheet and Cancel leaves the stake count unchanged in the database.

1.2 Chat: send clears the input and dismisses the keyboard; sender name above the message (frontend patch 0003)

Wrong. After sending, the input re-focused itself so the keyboard stayed up on phones, and the sender's name was rendered inline with the message.

Changed. `src/components/ChatInput.jsx`, `ChatPanel.jsx`, `ChatPanel.css`: the input is cleared and blurred on send; messages render the name above the text (TikTok-style layout).

Verified. Browser check: a sent message clears the field and drops focus; the name renders above the text.

1.3 End-of-battle result pop-up off by default (frontend patch 0005)

Changed. `src/core/constants.js` + `src/screens/Matches.jsx`: the modal is behind a build-time switch, `VITE_BATTLE_RESULT_MODAL=true` restores it. Requested by the owner ("disable the end-of-battle message entirely").

1.4 Round lock and server rejections made visible (frontend patch 0006)

Wrong. Server-side order rejections other than the guest case were dropped silently by the socket layer (`guestSocketError.js` only handled `GUEST_FORBIDDEN`), so a trader tapping into a freeze or a lock saw nothing happen.

Changed. New `src/live/services/tradeRejectionToast.js` shows a toast for `ROUND_LOCKED`, `BATTLE_FROZEN`, `ORDER_FREEZE`, `LAST_SECOND_SNIPE_LOCK`, throttling and insufficient balance. The new `battleLocked` socket event (see chapter 4) sets `matchStore.roundLock`; `TradingPanel.jsx` shows **LOCKED** on the buttons and disables them until the next round starts. The snapshot signature now includes `locked` and `trader_count` so those changes reach the store.

Verified. Browser smoke: buttons switch to LOCKED within 10 ms of the lock event; a locked order returns 409 `ROUND_LOCKED`.

1.5 Dependency hygiene (frontend patches 0001, 0002)

`eslint-plugin-react-hooks` 7.0.1 → 7.1.1 removes the ERESOLVE peer conflict with ESLint 10 so `npm ci` works without `--legacy-peer-deps`. The GitHub workflow no longer prints the production API and socket URLs into the CI log.

2. Admin app

2.1 Login / admin redirect loop (admin patch 0002)

Wrong. A non-admin login, or a half-restored session (token without user), bounced between `/login` and `/admin` forever.

Changed. `src/Pages/ProtectedRoute.jsx` reads user and token from the auth store; non-admin and incomplete sessions land on `/unauthorized`, which signs out before returning to login (`src/Pages/UnauthorizedPage/index.jsx`). Regression test `tests/admin-auth-routing.test.cjs` (12 checks).

2.2 Dashboard failed when Google Fonts was unreachable (admin patch 0004)

Wrong. `NpcSimulator.css` (and three other page stylesheets) `@import`ed Poppins from `fonts.googleapis.com`. When the font host is blocked (ad-blocker, firewall, offline) the lazy `/admin` chunk's CSS preload rejected and the `ErrorBoundary` replaced the whole dashboard with "React Application Error".

Changed. The font is a top-level `<link>` in `index.html` (preconnect + stylesheet); every Poppins declaration has a real fallback stack. Verified with requests to the font host aborted: the dashboard renders after login and the built CSS contains no `googleapis @import`.

2.3 Risk Terminal stale ticks and NPC status (admin patch 0001)

Wrong. The Risk Management Terminal kept showing the last socket tick after the round ended or the socket dropped, and the NPC Simulator page said "SIMULATOR ACTIVE" from the saved flag even when no worker was running.

Changed. `src/hooks/useRiskTerminalSocket.js` expires a tick after 5 s or on disconnect; the NPC page derives its status from `runningOnLeader` (`src/utils/npcRuntimeStatus.js`). Regression test `tests/npc-risk-regression.test.cjs` (16 checks).

2.4 CI (admin patch 0003)

The workflow no longer prints the production API URL; `npm ci` is used; `sw.js` and the manifest are uploaded with no-cache headers so PWA updates are not held back by the CDN.

2.5 NPC Simulator → Organic Engine tab (admin patch 0005) — see chapter 4

2.6 Market Impact & Sensitivity page (admin patch 0006) — see chapter 5

2.7 Risk Breaches, Risk Management Terminal, Price Movement Audit, Battle Inspector (admin patch 0007) — see chapter 6

3. Backend — cross-cutting fixes

3.1 CORS allow-lists and Swagger in production (backend patch 0003)

Wrong. The API reflected any origin and served Swagger UI in production.

Changed. `apps/api/src/common/utlis/cors.util.ts`, `main.ts`, `config/env.validation.ts`, `.env.example`: `CORS_ORIGIN` and `WS_CORS_ORIGIN` allow-lists (comma separated); with the variables empty the API still reflects any origin but warns at boot in production; Swagger is off in production unless `SWAGGER_ENABLED=true`. Jest spec `cors.util.spec.ts` (5 cases). Set `CORS_ORIGIN=https://ticktrade.app,https://dev.ticktrade.app,...` on the servers.

3.2 Risk-terminal tick uses the shared Redis (backend patch 0001)

Wrong. Buffered human stakes were read from the wrong Redis connection and counted as NPC volume on the Risk Terminal (a human BUY of 100 read as 0 real).

Changed. `apps/engine/src/battle/risk-terminal-tick.publisher.ts` uses `getSharedCommandRedis()`. Regression `risk-terminal-regression.spec.ts` (8 checks).

3.3 NPC worker honours the saved OFF flag on boot (backend patch 0002)

Wrong. A persisted STOP was forced back ON every time the worker restarted.

Changed. `apps/npc-worker/src/npc-worker.command.listener.ts` reads the durable flag on boot. Regression `npc-startup-off.regression.spec.ts`.

3.4 Email tests aligned with the documented contract (backend patch 0004)

Three stale specs asserted the opposite of the documented behaviour (camelCase event segments are valid stored mapping keys; HTML is master-wrapped and unresolved `{{tags}}` are stripped by design). The specs now assert the contract.

4. Organic NPC engine (backend patch 0005, admin patch 0005, frontend patch 0006)

This is the largest change and implements the *Master NPC Architecture and Risk Management Directives 2.1* and the *Live Testing & Artificial Reaction Flaw* memo.

4.1 What was wrong

Three separate systems reacted to every real order, instantly, which is what made the NPC layer feel artificial and made the admin settings look "fake":

Where	Behaviour
API, inside the user's own trade transaction (<code>trades.service.ts</code>)	NPC counter-chunks placed on the opposite side in the same transaction (<code>NPC_DOMINANCE_PROTECTION_ON_USER_ORDER</code> on by default), plus a last-moment hedge
npc-worker listener	"Reactive burst": 3–4 counter-trades within 5–8 s of any bet $\geq$ \$5k; emergency pot flips in the last 15 s
Legacy simulator loop (16,696 lines)	Whale-spam defence, late-instant defence, pot-parity servo, weak-side support, force-win at close

The 99 NPC settings were re-read by a 60 s poll with no push, several keys were dead, and the two main levers (pace, aggression) only re-tuned the reactive layers. Settlement split the real losers' pool between real winners **and** NPC winners; a hidden rule picked the side with the *smaller* real pot whenever both sides had real users; there was no round lock.

4.2 What changed

One engine, driven entirely by a versioned settings document, replaces all three (the old simulator remains as a live-switchable fallback, `npc_engine_mode = legacy`).

- **Settings** — `packages/shared/src/npc-organic/settings.ts`: one JSON document (`system_settings.npc_organic_settings` , versioned, with `updatedAtBy`) described by a field schema (54 fields; label, help, range, unit and an *applies* tag: instantly / next round / at settlement). The API validates against the schema, mirrors the document to Redis and pushes it on `engine:events:npc.organic.settings` ; the engine applies a push within one heartbeat, polls every 20 s as fallback and reads the database on boot. The admin form is rendered from `GET /admin/npc/organic/schema` , so a setting that does not exist in the engine cannot exist on the page.
- **Round flow** — `apps/engine/src/npc-organic/organic-npc-engine.ts`: no NPC trades at 0 ms; 1–5 NPCs (10–20 in peak hours) trickle in after a random 1–15 s delay; then arrivals follow a Poisson process whose rate ramps to a peak and fades late, with quiet periods and a short boost after a real trade (never sized against the trade). Whale / mid / retail tiers with admin stake ranges and human-looking ladder rounding. One side is drawn to lead heavily (60–90 % of NPC volume) for most of the round; no parity servo.
- **Defences** — every 5 s the engine projects the settlement; *Defense A* leans the organic flow against the projected exposure (`steeringStrengthPct`); at `T - finalInjectionLeadMs` it sets the round lock (`tt:battle:lock:<id>`), publishes `engine:events:battle.locked` , re-reads the exposure and executes one sliced whale injection: *Lightning Dilution* (bisection on the winning side until real winners are paid exactly the net real loser pool, plus a safety margin) or *Ghost Town* (all real

users on one side → inject on the opposite side by a margin). Orders after the lock are rejected with `ROUND_LOCKED` on all three validation paths (fast validation, full validation, inside the trade transaction).

- **Settlement** — `computeDirectiveSettlement` (shared) and `apps/engine/src/battle/directive-settlement.applier.ts`: net positions per user; $R = V_{rl} \times (1 - fee)$; $Pot = R + V_{sl}$; simulator profit capped at its own losing volume; surplus to real winners; company = commission (+ the unpaid pool in a ghost town). The invariants are written to the new `npc_round_audit` table (migration `20260913120000_add_npc_round_audit`).
- **Hosts** — the engine runs in the `npc-worker` (default) or in the engine process; `NpcEngineSwitch` runs exactly one of organic / legacy and switches live. In organic mode the API skips the in-transaction hedge/dominance and the async dominance follow-up; the worker ignores reactive bursts and align-at-close.
- **Admin** — NPC Simulator → *Organic Engine* tab: status bar (mode with a live organic ↔ legacy switch, engine heartbeat, "saved vN / engine applied vN"), the schema-driven settings form with unsaved markers and inline server validation, *Live round* telemetry every second, *Round audits* with the settlement invariants. Legacy-only tabs are labelled.
- **Trading app** — LOCKED buttons on `battleLocked`, rejection toasts (chapter 1.4).

4.3 Verification

Shared tests 27/27 (schema, plan maths, settlement invariants, injection solver); engine 8/8 (round lifecycle with a fake clock); browser smoke 30/30 with one real trader in a live round: the trickle continues after the real trade, the lock fires at the configured lead, LOCKED buttons within 10 ms, 409 on a locked order, settlement invariants hold, the stake settles, the admin form round-trips and the engine applies the saved version. Mode switch organic → legacy → organic verified live: the old simulator resumes on the next round.

4.4 Deploying

Run the Prisma migration, build shared → api / engine / npc-worker / settlement-worker, restart. No new environment variables (`NPC_ORGANIC_SETTINGS_POLL_MS` optional). The engine starts in organic mode with the directive defaults; the admin page switches to legacy at any time. Full operator notes: `docs/NPC-ORGANIC-ENGINE.md` in the backend repo.

Two things to know: Ghost Town can inject a large amount when the sole real user sits on the side the NPCs already lead (cap it with *Final injection max*); the single whale NPC occasionally fails a Redis balance reserve and that trade is skipped.

5. Market Impact & Sensitivity (backend patch 0006, admin patch 0006)

5.1 The question

A \$100,000 SELL was reported to move 16.6 pips against a 16.0-pip formula, and the difference was described as "normal, not exact every time". That is not correct for the formula path: with nothing else moving the price, a real-user trade moves **exactly** $\text{curve}(\text{amount}) \times (\text{sensitivity} \div 5) \times \text{multiplier}$ pips, at every price level. Measured with the NPC simulator stopped: \$100,000 → 16.0000 pips, \$50,000 → 8.0000, \$1,000 → 0.2 (0.16 rounded to the 5-decimal quote). The extra 0.6 pip came from NPC ticks landing in the same window as the chart reading (with the simulator on, the real \$100k order moved exactly 16.0 and an NPC trade 750 ms later added 2.4; the chart read 18.4).

5.2 Real defects fixed

1. Hidden price-scaled minimum step on small real trades.

`enforceImmediateDirectionalMove` still ran after the deterministic path for every trade, sized with the price-scaled pip and the book skew. Any real order whose formula move rounded to zero (under ~\$300 at the reported settings) moved 0.8 pips into a leaning book and nothing into a level one. Now: real trades use the formula only, with a flat one-quote-tick floor (0.1 pip) at every price level; the nudge runs only for NPC ticks, in the live engine and in the audit replay.

2. Price Movement Audit stored the wrong config.

Settlement runs in the settlement-worker process, which never loads the live config; every queued settlement stored the compiled-in defaults (log / 5 / 1) and a wrong expected end price. The engine leader now records the config the round was priced with (override included) when the round closes; settlement reads it, falls back to the durable Redis config, and uses the configured freeze seconds instead of a hard-coded 10.

3. Live Trade Log missed trades.

It was fed from the coalesced price-tick channel (one publish per 200 ms), was disabled under `LIVE_BATTLE_REDIS_ONLY`, and summed only the last 50 rows. New event `engine:events:trade.priced.v1` (one row per priced trade, never coalesced); the round total is measured on the price itself; real / NPC / other split; formula next to actual for every real trade.

4. Saves took up to 5 s to reach the engine.

They are pushed on `engine:events:market.sensitivity.v1` and applied immediately (verified: a trade 300 ms after saving already used the new multiplier).

5. Admin page.

Date Range filter did nothing; Search and Field Filter only filtered the current page; Reference Amount / Reference Pips were not editable; the override input accepted 0; the preview divided by a hard-coded 5; the audit list contained only failures so its pass rate was meaningless; the MULTIPLIER column showed the event multiplier; the replay rounded prices to cents. All corrected; new `GET /admin/market-sensitivity/preview` runs the engine's own function with the live price and the sensitivity in force.

5.3 Verification

New spec `price-impact-deterministic.spec.ts` (8): exact 16.0 pips for the reported case; identical pips at prices 0.9 ... 4.5; pots / pressure / elapsed / finalize have no effect; quote-tick floor; sensitivity and override factors; curves; replay equals live pricing. Workbench: NPC-off trades exact at \$10 ... \$100,000; push applied within 300 ms; NPC-on log separating the exact real move from NPC ticks; settlement snapshot correct; admin browser checks 20/20. The full review (`Market-Sensitivity-Review-2026-09-13.pdf`) has the evidence tables.

6. Admin risk and inspection tools (backend patch 0007, admin patch 0007)

The four pages under *Risk & Security* and *Trading & Battles* — **Risk breaches**, **Risk Management Terminal**, **Price Movement Audit Tool** and **Battle Inspector** — were audited tool by tool (every endpoint, every filter, every number on the page, against the settlement code that produces the money they describe) and then rewritten where the audit found them wrong. This chapter explains what was wrong, what the new design is, and what a developer will find in the code.

6.1 The root problem: four tools, four private settlement formulas

Every one of the four tools re-derived "what settlement did with the money" on its own, with its own copy of the legacy Model A formula:

- Risk Breaches recomputed every round on every request with `computePayouts`, gross (un-netted) real volumes and *today's* fee rate, and called the result "out of pocket" even when the house had kept the commission.
- Battle Inspector hard-coded a 30 % fee (`FEE = 0.3`) in one place and used the live fee in another, so the list and the detail of the same battle disagreed; it never applied hedge netting (`hedged_user_count` was hard-coded to 0) and it computed a "house PnL" that was the platform fee minus an injection that no longer exists.
- The Risk Management Terminal showed `company_net_position = platform fee`, a number that can never be negative, so the round that costs the house the most (real winners paid from NPC money) looked like income. Its "Thin Margin" badge compared the payout to the very pool it was paid from — a ratio that is always ≤ 1 by construction and never fired for the rounds that matter.
- None of them knew about the organic NPC engine. Since the organic engine settles with the directive model (chapter 4: zero net NPC profit, net positions per user, commission on real losers only), every organic round shown by these tools was valued with the wrong model.

The new design has **one record of the money per settled round** and every tool reads it:

- `packages/shared/src/risk/round-valuation.ts` — the `RoundValuation` record (model, winner, fee rate, commission and its source, real losing / winning volume, net

real loser pool, real winners' profit, house retained, simulator volumes / profit / surplus / net, the model's own company figure, `houseCashResult` = real money in – real money out, `shortfall` = real winners' profit paid beyond the net real loser pool, user counts, hedged users, refunds, and the legacy-only fields). Three builders: `valuationFromDirective`, `valuationFromModelA`, `valuationFromTie`, and `parseRoundValuation` (which also upgrades the first organic release's audit rows on read).

- **Settlement writes it** for every model: the directive applier (`directive-settlement.applier.ts`) stores it as the `npc_round_audit.settlement` invariants; the legacy and tie paths of `battle-settlement.service.ts` build it and upsert the same row *after* the financial transaction commits, so an audit write can never roll back a settlement. Rows are created with `mode: 'legacy'` when the engine never opened them.
- **The API reads it** through one service, `apps/api/src/admin/round-valuation.reader.ts`: stored valuation when present; otherwise a rebuild from the rows with the model the round was settled with (directive for organic rounds — the stored winner is forced so a changed pot rule cannot flip the round — Model A otherwise, with the legacy engine's own hedge rule), capped at 300 rebuilt rounds per request and flagged `valuationSource: 'recomputed'` so the page can say the fee rate is today's.

Two conventions used everywhere from now on: *real* means human stakes net of hedges (per-user matched volume for the directive model, fully hedged users for the legacy model) with RPC house bots excluded; *house cash result* is the only "did this round make or cost the house money" figure, and it is the same number on all four pages.

6.2 Risk breaches

Found. Breach ceiling computed with the legacy model for organic rounds; gross volumes (a \$301,000 BUY + \$120,310 SELL by the same user counted as \$301,000 lost); the current fee rate applied to old rounds; `breachRate` returned as a ratio and multiplied by 100 on the page only when it happened to be ≤ 1 ; the summary counted the current page, not the period; custom dates converted with `toISOString()` (shifted a day west of UTC) and read as UTC days; the status column came from a separate list gated by `riskTerminal.view`, fetched once (limit 200, clamped to 100) and never refreshed; *Details* never called the detail endpoint (it showed the list row as key/value pairs); the page never sent `scope`; the day ended at 23:59:59.000; the detail's user PnL ignored refunds and its NPC payouts were hard-coded to 0; a resolution could be set on an unsettled battle and saving without a note kept the previous note; the "Risk breaches" notification toggles had no producer at all.

Now. `risk-breach-monitor.service.ts` is valuation-driven: a breach is `shortfall > 0`; "cost the house cash" is `houseCashResult < 0`; "real users won" is `realWinnerUserCount > 0`. The period summary is computed over the whole window (newest 5,000 rounds, `truncated` when more), per model, with `breachRatePct` / `npcLossRatePct` / `companyFundsRatePct` as percentages of settled non-tie rounds. Rows

carry the model, the valuation source, real losing volume, commission, real winners' profit, shortfall, house cash result and the resolution (status, note, who, when) — no second request. Scope adds `company_funds_only`. Custom ranges accept exact instants (`start_at` / `end_at`, ISO 8601, taking precedence over the date strings), so the browser sends its own local day boundaries; the day ends at `.999`. The detail (`GET /admin/risk-breaches/:battleId`) works for every settled round (tie included) and returns the settlement breakdown, per-user rows with BUY / SELL / hedged / net side / net exposure / lost / refunded / payout credits and a cash PnL taken from the ledger (`payout + refund credits - everything staked`), NPC rows with the payout actually written to `npc_trades.payout`, and database cross-checks (ledger credits, stakes-won principal and profit, mismatch). Resolutions are accepted only for COMPLETED battles and an empty note clears the previous one. A new `RiskBreachAlertScannerService` (leader-elected, every 60 s) raises the `RISK_BREACH` admin alert the Notification Preferences toggles promise, one per breached round, honouring the in-app and e-mail toggles.

Page (`RiskBreaches.jsx`, rewritten): period presets (today / week / month / all / custom with the date picker), scope selector, server summary cards (breached rounds, total shortfall, rounds that cost the house, total house cash lost, period house cash result, commission, winners' profit, largest shortfall, latest breach, settled rounds, breach rate, rounds real users won, by model, rebuilt count) with the truncation note; a results table with model, real losing, commission, winners' profit, shortfall, house cash and resolution; *Details* opens the server audit; acknowledge / resolve / clear refresh the list and the detail.

6.3 Battle Inspector

Found. `LIVE` badge on the first row of every page regardless of status (and in the header); custom dates shifted a day; the deep-history modal always sent a period (a battle-id lookup was silently narrowed to today) and had no "all time"; fee hard-coded at 0.3 while settlement used the live fee; list and detail commission formulas disagreed; hedge netting off and `hedged_user_count` always 0; organic rounds valued with the legacy model; participants labelled with the side and status of their *first* stake (a user with a LOST BUY and a REFUNDED SELL showed as one row); a stake with a null user crashed the detail; the live winner for an ACTIVE round used the legacy rule (and a Redis reader that throws when Redis is down) even when the organic engine was playing; no CANCELLED branch; every stake and price tick of every battle on the page was loaded.

Now. `admin-battle-inspector.service.ts` rewritten. Settlement figures come from the valuation (model, valuation source, fee rate, commission and source, real losing volume, net real loser pool, real winners' profit, shortfall, house cash result, model company net, simulator net, NPC profit, paid to NPC winners, refunds, ledger cross-checks, a `profit_mismatch` between the record and the database). Volumes distinguish `real_money` (net of hedges by the round's own rule), `real_money_gross`, `npc_money` and `rpc_money` (house accounts, in the pots but not real). For an ACTIVE round the winning side is the rule the engine will actually apply — directive pot dominance when the organic engine plays the round, the legacy resolver otherwise — on the live Redis pots, with `winner_rule` saying which; the Redis reader can no longer throw. `all_participants` is one row per user **per side** with `is_hedged` and `is_rpc`. CANCELLED rounds get their

own settlement kind. The list adds `settlement_model`, `valuation_source`, commission, house cash and hedged users per row and accepts `start_at` / `end_at`. The deep-history thunk sends no period when a battle id is given, and the modal offers *All time* and formats picked days in local time.

Page (`BattleInspector.jsx`): LIVE only when the row's status is ACTIVE; period presets; MODEL / COMMISSION / HOUSE CASH columns; the detail shows gross vs net real money, hedged users, RPC stakes, the winning side with its rule, the NPC engine, a *Settlement* panel with the record's figures, and participants per side with hedged / house markers.

6.4 Price Movement Audit Tool

Found. Two flag rules under one threshold (intra-round swing % and expected-vs-actual variance %) so a calm round with a large swing failed and the page could not say why; freeze seconds were never stored (the replay assumed 10, the live engine used 0); per-round overrides were ignored by the historical config; the NPC re-run diverged from the live path (the live NPC path carries order-book state the replay cannot see), producing false FAILs on rounds where the real trades were priced exactly; `price_ticks.createdAt` was the batch flush time, not the trade time, and ticks were ordered by that alone; a half-written snapshot (config stored, expected price missing) when the replay threw; CSV ignored the status filter; the 5,000-row cap was silent; the status parameter was unvalidated; the threshold input was a text box; the post-save refetch dropped the status filter.

Now. One rule: a round **FAILS** when $|\text{actual end} - \text{expected end}| \div \text{start price}$ reaches the tolerance; rounds settled before the snapshot existed fall back to the swing check and say so (`flag_reason: 'swing'`, counted separately in the summary). The expected path is `replayBattlePricing(..., { actualNpcTicks })` in `packages/shared/src/battle/price-audit-replay.ts`: real trades are re-run through the pricing formula, NPC trades apply the price move actually recorded for them (`replay_method: formula_real_plus_actual_npc`, with `npcTicksMatched` / `npcTicksMissing`), so the variance isolates what the tool exists to check — whether real trades moved the price by exactly the admin formula. Settlement stores the snapshot atomically (config, freeze seconds, config source, replay method, matched / missing counts, expected price, variance) or not at all. `price_ticks` rows are stamped with the engine's event time and every reader orders by (`createdAt`, `id`). The list validates `status`, reports `truncated`, adds `config_source`, `freeze_seconds_used`, `replay_method`, `has_stored_snapshot` and `flag_reason`; CSV honours the status filter and carries the same columns; the replay endpoint reports the replayed end price next to the stored one, the config source and the tick sources. The page adds *All Time*, a numeric threshold input, a "swing" marker on rows without a stored verdict, freeze seconds and config source in the details, and the replay method under the chart; the "Avg Variance" card is labelled by what it actually averages.

Settlement also repairs the round extremes: `battles.roundHigh` / `roundLow` were seeded with the start price and only refreshed at close when the market state was

readable; the legacy path now folds the recorded price path in (as the directive path already did) and writes the corrected values back, so Model A scoring and the audit tools see the real range.

6.5 Risk Management Terminal

Found. `company_net_position` was the platform fee alone (never negative); "Thin Margin" was degenerate; the REST endpoint and the socket tick computed different numbers (no round high/low override, `price_ticks` vs the engine's market state); "NPC" pots included RPC / house stakes; the projection used the legacy model for organic rounds; an unbounded per-second projection with no in-flight guard; detection scan of "the last 200 completed battles" unordered; alerts written by the detection scanner were never shown anywhere; a new round was shown up to 15 s late; the injection endpoint carried dead code and a Swagger description that no longer matched what it does (it places a real trade as the house user, not an NPC trade).

Now. The engine tick (`risk-terminal-tick.publisher.ts`) resolves the NPC engine for the round and projects with the model that will settle it: `projectDirectiveScenarios` (the real `computeDirectiveSettlement` with the winner forced) for organic rounds, `computeDualScenarioProjection` for legacy ones — and in both, `company_net_position` **is now the house cash result** (real losing volume – real winners' profit; negative when real winners are paid from NPC money), with `shortfall`, `real_losing_volume`, and for the directive model the NPC profit / surplus / simulator net. Live rounds under `LIVE_BATTLE_REDIS_ONLY` buffering read the Redis stake and NPC-trade buffers (as the legacy live context did). One projection at a time (a slow tick is skipped). The last payload is kept in Redis (`ticktrade:risk-terminal:last_tick`, 10 s) and `GET /admin/risk-terminal/live-state` serves it (`source: engine_tick`), computing the same projection itself only when the engine has not published for the round (`source: api_compute`). New `GET /admin/risk-terminal/alerts` lists the detection scanner's unexpired alerts; the scanner now looks at the newest rounds first. The payload contract lives in `packages/shared/src/events/risk-terminal-tick.v1.ts`.

Page (`RiskManagementTerminal/index.jsx`): settlement model, leading side and reason, current price and data source; per scenario *House Cash Result*, *Commission*, *Shortfall* (*beyond real pool*), real winner volume and, for organic rounds, NPC profit after cap / surplus to real; *Beyond real pool / House pays* badges replace "Thin Margin"; a Trading Integrity Alerts table; the page refetches at once when the socket tick names a new round.

6.6 Shared building blocks added

- `packages/shared/src/npc-organic/round-context.ts` — `resolveNpcEngineContext` (which engine plays a round: audit row → Redis → `system_settings` → default, with the organic settings) and `buildDirectiveRoundInput` (the directive model's view of a round from the database or the live Redis buffers). Used by the engine tick, the terminal REST fallback and the Battle Inspector; settlement keeps its transactional copy.

- `packages/shared/src/npc-organic/directive-settlement.ts` — `forceWinnerSide` on the round input (`winnerReason: 'forced_scenario'`), `projectDirectiveScenarios` and the terminal-shaped `DirectiveScenarioProjection`.
- `packages/shared/src/payout/battle-payout.scenarios.ts` — `model`, `house_cash_result`, `real_losing_volume`, `shortfall` on the legacy scenario; `company_net_position` redefined as the house cash result.
- `packages/shared/src/battle/price-audit-replay.ts` — `ReplayOptions.actualNpcTicks`, `tick source`, `ReplayResult.method` / `npcTicksMatched` / `npcTicksMissing`; the historical freeze default now matches the engine (0).
- `apps/api/src/admin/round-valuation.reader.ts`, `risk-breach-alert-scanner.service.ts`; rewritten `risk-breach-monitor.service.ts`, `admin-battle-inspector.service.ts`, `risk-terminal.service.ts`, `price-movement-audit.service.ts`.

6.7 Other defects found on the way

- **Duplicate stake ledger rows** (`apps/api/src/trades/ledger-async-writer.service.ts`): `enqueue` pushed each row to the local queue *and* to Redis, one flush drained both, and the existence check only saw committed rows — so every stake produced two identical ledger lines (same `referenceId`, same `balanceAfter`; the balance itself was debited once). The batch is now de-duplicated by reference. Existing duplicates are harmless audit noise; a one-off `DELETE` of the later duplicate per (`referenceId`, `type='stake'`) cleans them.
- **Admin build**: the main bundle passed the 2 MiB workbox precache default and the service-worker build failed; `vite.config.js` raises the limit to 4 MiB.

6.8 Compatibility and things deliberately left alone

- No database migration: the record lives in the existing `npc_round_audit.settlement` JSON column. Rounds settled before this release are rebuilt on read (marked *rebuilt* on the pages, today's fee rate); the first organic release's audit rows are upgraded on read without a rebuild.
- API responses keep their previous field names where the pages already used them and add fields; the only renamed meaning is `company_net_position` in the terminal payload (now the house cash result — documented in Swagger and on the page). Old `breachRate` / `npcLossRate` ratios are replaced by `breachRatePct` / `npcLossRatePct`.
- The socket room `admin_risk_terminal` is still joined by *role* (`ADMIN`) rather than by the `riskTerminal.view` permission, because the `WebSocket` module cannot import the admin permission resolver without a circular dependency; the REST endpoints are permission-gated. Worth a follow-up in the gateway.
- `risk_breach_resolutions.resolvedByAdminId` cascades on admin deletion (deleting an admin deletes their review marks). Changing it to `SET NULL` needs a

Prisma migration; not done here.

- The company-exposure alert thresholds and the detection heuristics (multi-accounting by shared payout account) are unchanged.

6.9 Verification

New specs: `risk/round-valuation.spec.ts` (5), `npc-organic/directive-scenarios.spec.ts` (3), `npc-organic/round-context.spec.ts` (2), `battle/price-audit-replay-hybrid.spec.ts` (3); the engine's risk-terminal regression harness extended with legacy-vs-organic projection cases (10). Suites: shared 232/235 (the same 3 pre-existing `last-moment-company-hedge` failures), engine 85/97 (12 pre-existing pricing / legacy-protection failures; one previously failing terminal case now passes). Workbench: a round with a real user hedged \$500 of a \$3,500 BUY settled with the record `realLosingVolume 3000 · commission 900 · netRealLoserPool 2100 · houseCashResult 3000 · shortfall 0 · hedgedUserCount 1`; the ledger shows the \$500 matched refund on both sides and the breach detail's cash PnL is $-\$3,000$; the terminal serves the engine tick for the live round (`model: directive`); the audit replay of a 95-trade NPC round reproduces the stored expected price exactly (95 recorded NPC moves matched, variance 0); browser checks over the four pages 14/14 (`tools/admin-tools-smoke.cjs`).

7. TickTrade Intelligence Core (the owner's local tool)

7.1 Version 3.6.0

The tool's ZIP importer failed on archives that still contained `node_modules` or Finder metadata ("invalid or has too many entries"). Dependency, build, VCS and Finder folders are now ignored before the limits apply, on both the baseline importer and the Studio working-version extractor, with precise messages. Every swallowed exception lands in `data/logs/errors.log` with the error type. Four parallel code reviews produced 29 findings; 21 fixed with tests, 8 documented. Python 562/562, node 13/13.

7.2 Version 3.6.2

Start failed with "dependency postgres failed to start ... exited (1)" because a Docker volume from a previous workspace (created for another compose project) was reused. The tool now reads the dead container's log and explains it, and offers two maintenance actions: *Reset local database* (removes the project's database volume, with a confirmation) and *Free Docker space*. The immediate manual fix is `docker volume rm <project>_database` then *Start*.

8. Deploying, testing and known pre-existing failures

8.1 Order of deployment

1. Backend: `pnpm --filter @ticktrade/shared prisma:migrate:deploy` (adds `npc_round_audit`), build `shared`, then `api` / `engine` / `npc-worker` / `settlement-worker`; restart all five processes (the settlement worker must restart too — it reads the per-round pricing record and now writes the round valuation; the engine publishes the new terminal tick; the API runs the breach alert scanner). Patch 0007 needs no migration.
2. Admin: build and deploy `dist`.
3. Trading app: build and deploy `dist`.
4. Optional environment: `CORS_ORIGIN`, `WS_CORS_ORIGIN`, `SWAGGER_ENABLED` (chapter 3.1); `TRADE_PRICED_PG_PERSIST=0` disables the per-trade log rows if ever needed.

8.2 Test matrix

Suite	Result	Notes
shared (<code>node --test</code>)	232 pass / 3 fail	the 3 failures (<code>last-moment-company-hedge.spec.ts</code>) fail on the 12 Sep drop before any change
engine (<code>node --test</code>)	85 pass / 12 fail	the 12 failures (pricing model specs + legacy protection specs) fail on the 12 Sep drop before any change; one terminal harness case that failed on the drop now passes
new specs	npc-organic 27 + organic engine 8 + price impact 8 + round valuation 5 + directive scenarios 3 + round context 2 + hybrid replay 3 + terminal harness 10, all pass	
admin regressions	auth routing 12/12, npc-risk 16/16, fonts 2/2	<code>tests/*.cjs</code>
browser smokes	14/14 (general), 30/30 (organic engine), 20/20 (market sensitivity), 14/14 (risk & inspection tools)	<code>tools/*.cjs</code> in the workbench

8.3 Pre-existing failures left untouched

The 12 engine and 3 shared failures listed above concern the legacy pricing-model and last-moment hedge specs. They fail identically on the untouched 12 September sources and are outside the scope of the delivered changes; they are listed so nobody mistakes them for regressions.